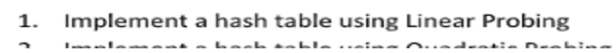
LAB 4

Hashing Programs in C

1. Implement a hash table using Linear Probing

Question 1 Screenshot



ALGORITHM

Algorithm:

1. Create a hash table of fixed size and initialize all positions to EMPTY.

2. Define the hash function as key % TABLE_SIZE.

3. For each key, calculate its initial hash index.

4. If the position is occupied, move to the next position using (index + 1) % TABLE_SIZE.

5. Continue until an empty position is found.

6. Insert the key into that position.

7. Display the final hash table.

C PROGRAM

```
#include <stdio.h>

#define SIZE 10
#define EMPTY -1

int hash(int key)
{
    return key % SIZE;
}

void insert(int table[], int key)
{
    int index = hash(key);
    int start = index;

    while (table[index] != EMPTY)
    {
        index = (index + 1) % SIZE;

        if (index == start)
        {
            printf("Hash table is full.\n");
            return;
        }
    }

    table[index] = key;
}

void display(int table[])
{
    int i;

    printf("\nHash Table:\n");
    for (i = 0; i < SIZE; i++)
    {
        if (table[i] == EMPTY)
            printf("Index %d : EMPTY\n", i);
        else
            printf("Index %d : %d\n", i, table[i]);
    }
}
```

```
int main()
{
    int table[SIZE];
    int keys[] = {23, 43, 13, 27};
    int n = 4;
    int i;

    for (i = 0; i < SIZE; i++)
        table[i] = EMPTY;

    for (i = 0; i < n; i++)
        insert(table, keys[i]);

    display(table);

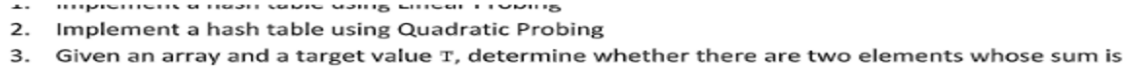
    return 0;
}
```

OUTPUT SCREENSHOT

```
Sample Output
Hash Table:
Index 0  :  EMPTY
Index 1  :  EMPTY
Index 2  :  EMPTY
Index 3  :  23
Index 4  :  43
Index 5  :  13
Index 6  :  27
Index 7  :  EMPTY
Index 8  :  EMPTY
Index 9  :  EMPTY
```

2. Implement a hash table using Quadratic Probing

Question 2 Screenshot



ALGORITHM

Algorithm:

1. Create a hash table of fixed size and initialize all positions to EMPTY.

2. Define the hash function as key % TABLE_SIZE.

3. For each key, calculate the initial hash index.

4. If the position is occupied, use quadratic probing:

index = (hash + i*i) % TABLE_SIZE, where i = 1, 2, 3, ...

5. Continue until an empty position is found.

6. Insert the key into that position.

7. Display the final hash table.

C PROGRAM

```
#include <stdio.h>

#define SIZE 10
#define EMPTY -1

int hash(int key)
{
    return key % SIZE;
}

void insert(int table[], int key)
{
    int h = hash(key);
    int index;
    int i;

    for (i = 0; i < SIZE; i++)
    {
        index = (h + i * i) % SIZE;

        if (table[index] == EMPTY)
        {
            table[index] = key;
            return;
        }
    }

    printf("Could not insert %d.\n", key);
}

void display(int table[])
{
    int i;

    printf("\nHash Table:\n");
    for (i = 0; i < SIZE; i++)
    {
        if (table[i] == EMPTY)
            printf("Index %d : EMPTY\n", i);
        else
            printf("Index %d : %d\n", i, table[i]);
    }
}

int main()
{
    int table[SIZE];
```

```
    int keys[] = {23, 43, 13, 27};
    int n = 4;
    int i;

    for (i = 0; i < SIZE; i++)
        table[i] = EMPTY;

    for (i = 0; i < n; i++)
        insert(table, keys[i]);

    display(table);

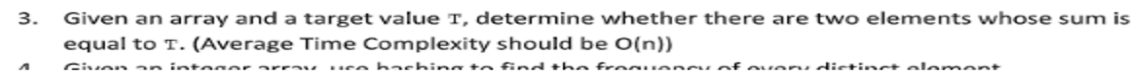
    return 0;
}
```

OUTPUT SCREENSHOT

```
Sample Output
Hash Table:
Index 0  :  EMPTY
Index 1  :  EMPTY
Index 2  :  EMPTY
Index 3  :  23
Index 4  :  EMPTY
Index 5  :  43
Index 6  :  EMPTY
Index 7  :  EMPTY
Index 8  :  13
Index 9  :  27
```

3. Given an array and a target value T, determine whether there are two elements whose sum is equal to T. (Average Time Complexity should be O(n))

Question 3 Screenshot


3. Given an array and a target value T, determine whether there are two elements whose sum is equal to T. (Average Time Complexity should be O(n))
4. Given an integer array, use hashing to find the frequency of every distinct element

ALGORITHM

Algorithm:

1. Create an empty hash set.
2. Traverse the array from left to right.
3. For the current element x, calculate complement = T - x.
4. Check whether complement is already present in the hash set.
5. If it is present, a pair with sum T exists; print the pair and stop.
6. Otherwise, insert x into the hash set.
7. If the complete array is traversed without finding a pair, report that no pair exists.
8. Average time complexity: O(n).

C PROGRAM

```
#include <stdio.h>

#define SIZE 100

int main()
{
    int a[SIZE], n, T;
    int hash[SIZE * 2];
    int i, x, complement;
    int found = 0;

    for (i = 0; i < SIZE * 2; i++)
        hash[i] = 0;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter array elements:\n");
    for (i = 0; i < n; i++)
        scanf("%d", &a[i]);

    printf("Enter target value T: ");
    scanf("%d", &T);

    for (i = 0; i < n; i++)
    {
        x = a[i];
        complement = T - x;

        if (complement >= 0 && complement < SIZE * 2 &&
            hash[complement] == 1)
        {
            printf("Pair found: %d + %d = %d\n",
                   complement, x, T);
            found = 1;
            break;
        }

        if (x >= 0 && x < SIZE * 2)
            hash[x] = 1;
    }

    if (!found)
        printf("No pair found.\n");
    return 0;
```

```
}
```

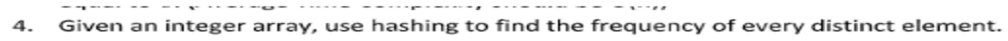
OUTPUT SCREENSHOT

```
Sample Output

Enter number of elements: 5
Enter array elements:
2 7 11 15 3
Enter target value T: 10
Pair found: 3 + 7 = 10
```

4. Given an integer array, use hashing to find the frequency of every distinct element.

Question 4 Screenshot



ALGORITHM

Algorithm:

1. Create a hash table to store each distinct element and its frequency.
2. Traverse every element of the array.
3. Calculate the hash index for the element.
4. If the element is already present, increment its frequency.
5. Otherwise, insert the element with frequency 1.
6. After processing all elements, traverse the hash table.
7. Display each distinct element along with its frequency.
8. Average time complexity: O(n).

C PROGRAM

```
#include <stdio.h>

#define SIZE 20
#define EMPTY -1

typedef struct
{
    int key;
    int frequency;
} HashEntry;

int hash(int key)
{
    if (key < 0)
        key = -key;
    return key % SIZE;
}

void insert(HashEntry table[], int key)
{
    int index = hash(key);
    int start = index;

    while (table[index].key != EMPTY)
    {
        if (table[index].key == key)
        {
            table[index].frequency++;
            return;
        }

        index = (index + 1) % SIZE;

        if (index == start)
            return;
    }

    table[index].key = key;
    table[index].frequency = 1;
}

int main()
{
    int a[] = {10, 20, 10, 30, 20, 10, 40, 30};
    int n = 8;
```

```
    HashEntry table[SIZE];
    int i;

    for (i = 0; i < SIZE; i++)
    {
        table[i].key = EMPTY;
        table[i].frequency = 0;
    }

    for (i = 0; i < n; i++)
        insert(table, a[i]);

    printf("Element\tFrequency\n");
    for (i = 0; i < SIZE; i++)
    {
        if (table[i].key != EMPTY)
            printf("%d\t%d\n",
                   table[i].key, table[i].frequency);
    }

    return 0;
}
```

OUTPUT SCREENSHOT

```
Sample Output

Element Frequency
20      2
40      1
10      3
30      2
```